

File-Based Routing in Next.js

Beginner → Intermediate → Expert | Interview Q&A with Answers

Akshay Chavhan | Target: Syngenta India + All Companies | April 2026

■ BEGINNER LEVEL

What is File-Based Routing?

In React you **manually configure** routes using React Router with JSX. In Next.js, the **folder and file structure itself IS the routing** — no configuration needed at all. Creating a file automatically creates a route.

```
// React - manual routing (you write this every time)
<Routes>
  <Route path="/"          element={<Home />} />
  <Route path="/about"      element={<About />} />
  <Route path="/products"   element={<Products />} />
</Routes>

// Next.js - just create the file, route exists automatically
app/page.tsx           → /
app/about/page.tsx     → /about
app/products/page.tsx  → /products
```

■ Rule: Every folder = URL segment. page.tsx inside = the UI rendered at that route.

Basic Route Mapping — Folder → URL

```
app/
■■■ page.tsx           → /           (home)
■■■ about/
■   ■■■ page.tsx       → /about
■■■ contact/
■   ■■■ page.tsx       → /contact
■■■ products/
    ■■■ page.tsx       → /products
```

Special Reserved Files in App Router

These filenames have special meaning — Next.js treats them differently from regular components:

File	Purpose	Auto-creates
page.tsx	UI for that route — makes it publicly accessible	The actual page
layout.tsx	Wraps children — persists across navigation without re-render	Shared shell
loading.tsx	Shown while page is fetching/rendering	Suspense boundary
error.tsx	Shown when component throws an error	Error boundary

not-found.tsx	Custom 404 for this route segment	notFound() handler
template.tsx	Like layout but creates fresh instance on every navigation	Re-renders always

```

app/
■■■ layout.tsx      ← Wraps EVERYTHING (root layout)
■■■ page.tsx        → / (home)
■■■ loading.tsx     ← Shows spinner while / loads
■■■ error.tsx       ← Shows if / crashes
■■■ not-found.tsx   ← Custom 404
■■■ dashboard/
  ■■■ layout.tsx    ← Wraps ONLY /dashboard/* routes
  ■■■ page.tsx      → /dashboard
  ■■■ loading.tsx   ← Shows while /dashboard loads
  ■■■ settings/
    ■■■ page.tsx    → /dashboard/settings

```

■ INTERMEDIATE LEVEL

1. Dynamic Routes — [param]

When one page needs to handle many different URLs (product pages, user profiles, blog posts), use square brackets **[param]** as a folder name. Next.js passes the value as **params** prop automatically.

```

app/
■■■ products/
  ■■■ page.tsx      → /products      (list all)
  ■■■ [id]/
    ■■■ page.tsx    → /products/1
                    /products/99
                    /products/wheat-hybrid ← Syngenta!

```

```

// app/products/[id]/page.tsx
export default async function ProductPage({
  params,
}): {
  params: { id: string };
}) {
  const product = await fetch(`/api/products/${params.id}`)
    .then(r => r.json());

  return (
    <div>
      <h1>{product.name}</h1>
      <p>ID: {params.id}</p>
    </div>
  );
}

```

2. Nested Dynamic Routes

```

app/
  shop/
    [category]/
      [productId]/
        page.tsx → /shop/electronics/iphone-15
                  /shop/seeds/wheat-hybrid-50

// params has BOTH values automatically
export default function ProductPage({
  params,
}: {
  params: { category: string; productId: string };
}) {
  // URL: /shop/seeds/wheat-hybrid-50
  // params.category = "seeds"
  // params.productId = "wheat-hybrid-50"
  return <div>{params.category} → {params.productId}</div>;
}

```

3. Catch-All Routes — [...slug] and [[...slug]]

Syntax	Matches	slug value	Matches root /docs?
[slug]	/docs/intro only (1 segment)	"intro" (string)	No
[...slug]	/docs/a, /docs/a/b/c	["a"] or ["a","b","c"] (array)	No
[[...slug]]	/docs, /docs/a, /docs/a/b	undefined or array	Yes

```

// app/docs/[...slug]/page.tsx
export default function DocsPage({
  params,
}: {
  params: { slug: string[] }; // Always an array
}) {
  // URL: /docs/api/v2/users
  // params.slug = ["api", "v2", "users"]
  return <div>Path: {params.slug.join(" / ")}</div>;
}

```

4. Route Groups — (groupName)

Folders wrapped in **parentheses ()** are completely invisible to the URL. They only exist for code organisation and applying different layouts to different sections of your app.

```

app/
  (marketing)/
    layout.tsx ← Full-width layout (no sidebar)
    about/page.tsx → /about (NOT /marketing/about)
    pricing/page.tsx → /pricing
  (dashboard)/
    layout.tsx ← Layout WITH sidebar
    home/page.tsx → /home
    settings/page.tsx → /settings

// Both layouts coexist – clean URLs, different shells ■

```

■ Use Case: At Syngenta — (public) for landing/marketing pages (no auth required), (portal) for agri-dashboard (requires auth + sidebar). Same app, different layouts, clean URLs.

5. Parallel Routes — @folder

Render **multiple independent pages simultaneously** in one layout — like a dashboard showing analytics and team panels side by side, each loading independently.

```
app/dashboard/
■■■ layout.tsx           ← Receives named slots as props
■■■ page.tsx             ← Main content (@children)
■■■ @analytics/
■   ■■■ page.tsx        ← analytics slot
■■■ @team/
    ■■■ page.tsx        ← team slot

// app/dashboard/layout.tsx
export default function DashboardLayout({
  children,
  analytics, // ← @analytics slot
  team,      // ← @team slot
}): {
  children: React.ReactNode;
  analytics: React.ReactNode;
  team: React.ReactNode;
}) {
  return (
    <div className="grid">
      <div>{children}</div>
      <div>{analytics}</div> /* Loads independently */
      <div>{team}</div>
    </div>
  );
}
```

6. Intercepting Routes — (.) (..) (...)

Show a route inside the current layout/context without full navigation. Classic example: clicking a photo in a feed shows it in a modal overlay (URL changes but you stay on the feed page). Navigate directly = full page.

```
app/
■■■ feed/page.tsx      → /feed (photo grid)
■■■ photo/
    ■■■ [id]/page.tsx  → /photo/123 (standalone full page)
    ■■■ (.)photo/      ← Intercepts /photo/* FROM /feed
        ■■■ [id]/page.tsx → Shows as MODAL over /feed

// (.) = intercept same level
// (..) = intercept one level up
// (...) = intercept from root
```

■ EXPERT LEVEL

Navigation — Link vs useRouter vs redirect

Method	Use In	When to Use	Notes
Link component	Server + Client	Declarative nav, anchor tags	Preloads on hover. Preferred.
router.push()	Client only	Programmatic action after	Import from 'next/navigation'
redirect()	Server only	Server redirect before render	Cannot use in Client Components
router.replace()	Client only	Nav without browser history	Login redirects
router.refresh()	Client only	Re-fetch server data	No full page reload

```
// 1. <Link> – preferred for all declarative navigation
import Link from 'next/link';

export default function Nav() {
  return (
    <nav>
      <Link href='/about'>About</Link>
      <Link href={`/products/${id}`}>Product</Link>
      <Link href='/login' replace>Login</Link>          {/* No history entry */}
      <Link href='/admin' prefetch={false}>Admin</Link> {/* No preload */}
    </nav>
  );
}

// 2. useRouter – programmatic navigation (Client Component only)
'use client';
import { useRouter } from 'next/navigation'; // App Router
// NOT 'next/router' – that is Pages Router!

export default function LoginForm() {
  const router = useRouter();
  const handleLogin = async () => {
    await loginUser();
    router.push('/dashboard'); // Go forward
    router.replace('/dashboard'); // Replace history
    router.back(); // Go back
    router.refresh(); // Re-fetch server component data
  };
}

// 3. redirect() – server-side redirect
import { redirect } from 'next/navigation';

async function ProfilePage({ params }: { params: { id: string } }) {
  const user = await getUser(params.id);
  if (!user) redirect('/not-found'); // Before rendering
  return <div>{user.name}</div>;
}
```

Reading Route Info — usePathname, useSearchParams, useParams

```
'use client';
import { usePathname, useSearchParams, useParams } from 'next/navigation';

export default function RouteInfo() {
  const pathname = usePathname();
  // /products/123 → '/products/123'

  const params = useParams();
  // app/products/[id]/page.tsx + URL /products/123
  // → { id: '123' }

  const searchParams = useSearchParams();
  // /products?sort=price&page=2
  // → searchParams.get('sort') = 'price'
  // → searchParams.get('page') = '2'

  return (
    <div>
      <p>Path: {pathname}</p>
      <p>ID: {params.id as string}</p>
      <p>Sort: {searchParams.get('sort')}</p>
    </div>
  );
}
```

generateStaticParams — Pre-build Dynamic Routes at Build Time

Tell Next.js which dynamic route values to pre-render as static HTML at build time. This is the App Router equivalent of **getStaticPaths** in Pages Router.

```
// app/products/[id]/page.tsx

// Runs at BUILD TIME — tells Next.js which [id]s to pre-render
export async function generateStaticParams() {
  const products = await fetch('https://api.example.com/products')
    .then(r => r.json());

  return products.map((product: { id: string }) => ({
    id: product.id, // Pre-renders /products/1, /products/2, ...
  }));
}

// dynamicParams = true (default): unknown IDs → SSR on first hit, then cached
// dynamicParams = false:          unknown IDs → 404
export const dynamicParams = true;

export default function ProductPage({ params }: { params: { id: string } }) {
  return <div>Product: {params.id}</div>;
}
```

■ Critical Gotchas

■ Gotcha 1: `useRouter` import is DIFFERENT per router. App Router → import from 'next/navigation'. Pages Router → import from 'next/router'. Mixing these = runtime crash, a very common interview trick question!

■ ■ Gotcha 2: A folder alone does NOT create a route. `app/dashboard/` folder exists but there's NO route until you add `page.tsx` inside it. `layout.tsx`, `loading.tsx`, `error.tsx` alone don't create accessible routes.

■ ■ Gotcha 3: Route groups (parentheses) are invisible to the URL but DO affect which `layout.tsx` applies. Two sibling route groups can each have their own layout with completely different UI shells.

■ ■ Gotcha 4: In Next.js 15+, `params` is a Promise. You must await `params` before accessing its properties: `const { id } = await params;` — skipping `await` = undefined values.

■ Best Practice: Use the `Link` component for navigation in JSX (it prefetches on hover automatically). Use `useRouter` only when you need programmatic navigation after an async action like form submit or API call.

■ INTERVIEW Q&A WITH DETAILED ANSWERS

Beginner Level

BEGINNER

Q1. What is file-based routing in Next.js and how is it different from React?

Answer: In Next.js, the folder and file structure automatically defines routes — no configuration needed. A file at `app/about/page.tsx` becomes the `/about` route automatically. In React, you manually define every route using React Router's and components. Next.js's approach reduces boilerplate and makes the routing structure visually clear in the file tree.

BEGINNER

Q2. What files have special meaning in the App Router? Name at least 4.

Answer: `page.tsx`: the UI for that route (makes it accessible). `layout.tsx`: wraps children, persists across navigation without re-render. `loading.tsx`: auto Suspense boundary shown during loading. `error.tsx`: auto Error boundary shown when component throws. `not-found.tsx`: custom 404 page for that segment. `template.tsx`: like `layout` but creates a fresh instance on every navigation.

BEGINNER

Q3. Does creating a folder in `app/` automatically create a route?

Answer: No! A folder alone does NOT create a route. You must add `page.tsx` inside the folder to make it a publicly accessible route. `layout.tsx`, `loading.tsx`, and `error.tsx` alone are special files but don't create addressable routes by themselves.

Intermediate Level

INTERMEDIATE

Q4. What are dynamic routes? How do you access the dynamic value?

Answer: Dynamic routes use `[param]` folder syntax to handle multiple URLs with one file. Example: `app/products/[id]/page.tsx` handles `/products/1`, `/products/99`, etc. Next.js passes the value via the `params` prop automatically: `{ params: { id: string } }`. For nested dynamic routes like `[category]/[productId]`, `params` contains both values.

INTERMEDIATE**Q5. What is the difference between [...slug], [slug], and [[...slug]]?**

Answer: [slug] = exactly one URL segment (/blog/post-1). [...slug] = one or more segments as an array (/docs/api/v2 → ['api','v2']). [[...slug]] = optional catch-all — also matches the root path with slug as undefined. Use [...slug] for documentation sites, [[...slug]] when root path also needs to match.

INTERMEDIATE**Q6. What are Route Groups? Why would you use them?**

Answer: Route Groups use (folderName) syntax — completely invisible to the URL but useful for code organisation and layout separation. Example: (marketing) and (dashboard) can each have their own layout.tsx, giving different UI shells for different sections. Routes inside remain clean: /about, /home — not /marketing/about.

INTERMEDIATE**Q7. What is the difference between useRouter from 'next/navigation' vs 'next/router'?**

Answer: next/navigation is for the App Router (Next.js 13+). next/router is for the legacy Pages Router. They have different APIs — App Router's useRouter has push, replace, back, refresh. Pages Router's router has push, replace, query object. Mixing imports is a very common bug — importing from the wrong package causes runtime errors.

INTERMEDIATE**Q8. What is generateStaticParams and when do you use it?**

Answer: generateStaticParams is the App Router equivalent of getStaticPaths. It runs at build time and returns an array of param objects — Next.js pre-renders a static HTML page for each. Use it for dynamic routes with known values (product pages, blog posts). Set dynamicParams=true (default) to allow unknown values to SSR on first request then cache.

Expert / Syngenta-Style Questions

EXPERT**Q9. What are Parallel Routes and when would you use them in a real project?**

Answer: Parallel Routes use @folder syntax and allow rendering multiple independent pages simultaneously in the same layout. Each slot loads independently with its own loading/error states. Real use case: a Syngenta agri-dashboard showing @weatherPanel, @cropPrices, and @alerts side by side — each fetching different data independently, so a slow weather API doesn't block the crop prices panel from rendering.

EXPERT**Q10. Explain Intercepting Routes and give a real example.**

Answer: Intercepting Routes display a route inside the current context without full navigation. Syntax: (.) for same level, (..) for one level up, (...) for root. Example: Instagram-style image grid — clicking a photo at /feed/photo/123 shows a modal overlay while the URL changes to /photo/123. Direct navigation to /photo/123 shows the full standalone page. This gives shareable URLs AND modal UX simultaneously.

SYNGENTA

Q11. Design the complete routing structure for Syngenta's product portal (crops, seeds, pesticides).

Answer: `app/(marketing)/page.tsx` → `/` | `app/(marketing)/about/page.tsx` → `/about` | `app/(portal)/layout.tsx` → Authenticated layout with sidebar | `app/(portal)/products/page.tsx` → `/products` (all) | `app/(portal)/products/[category]/page.tsx` → `/products/seeds` | `app/(portal)/products/[category]/[productId]/page.tsx` → `/products/seeds/wheat-hybrid-50` | `generateStaticParams` pre-renders known products. Route groups separate public vs auth sections. Nested dynamic routes give clean SEO-friendly URLs.

EXPERT

Q12. How does Link prefetching work in Next.js and when would you disable it?

Answer: Next.js automatically prefetches pages linked with the Link component when they enter the viewport in production. For SSG/ISR pages, the static payload is prefetched. For SSR pages, only the loading state prefetches. Disable with `prefetch={false}` for: rarely-visited admin pages, pages behind auth (no point prefetching if user can't access), or large pages that would waste bandwidth.